SECURE ROM HAL

Complete User & Developer Guide

A practical manual for building, testing, understanding, and publishing a C/HAL + AXI4-Lite RTL learning project

Guide focus	Host build, C regression, RTL simulation, driver architecture, troubleshooting, GitHub release hygiene
Primary tools	C11, CMake, CTest, Icarus Verilog, vvp, Git
Hardware status	Host-side tests and RTL simulation; physical board integration is not assumed
Security status	Educational secure-boot-related modules; no certification or production security claim implied

Documentation note

This guide is based on the described repository layout and project history. Before publishing, compare every path, register definition, status code, and command against the exact GitHub commit. Where behavior depends on implementation details, the guide tells you to inspect the current source rather than treating assumptions as verified facts.

Prepared for public project documentation

How to navigate this manual

The detailed manual is organized as 20 chapters with focused reference pages. Use the early chapters to set up the environment, the middle chapters to understand software and RTL layers, and the final chapters to automate verification and publish responsibly.

01. Project orientation

What this repository is • What the project demonstrates • What it does not prove • Repository map at a glance • How to use this guide • Recommended reading order • Development philosophy • Terminology used • Known project assumptions • Baseline evidence checklist

02. Host setup and prerequisites

Linux shell baseline • Required toolchain • Install packages on Ubuntu • Check the compiler • Check CMake and CTest • Check Icarus Verilog • Verify the repository path • Network-independent workflow • Permission and shell errors • Capture the environment

03. Clone, inspect, and first build

Clone or open the repository • Inspect top-level files • Read CMakeLists.txt first • Configure a clean build • Compile the project • Run CTest • Use verbose build output • Clean reconfigure safely • First-pass success criteria • Record your first baseline

04. Build system and source organization

CMake target model • C standard selection • Include directory discipline • Executable versus library • Test target separation • Linker diagnostics • Compiler warning policy • Build artifacts and Git • Adding a new test • Build-system review checklist

05. Platform types, status, and registers

Platform type layer • Status codes as API contracts • Register read helper • Register write helper • Set and clear bit operations • Volatile is not synchronization • Register address validation • Endianness and width • Host model versus MMIO • Platform utility test plan

06. GPIO HAL

GPIO API responsibilities • GPIO initialization • Pin range checks • Write and read semantics • Toggle implementation • GPIO status handling • GPIO test cases • GPIO simulation boundaries • Common GPIO debugging • GPIO extension exercises

07. UART and concurrency

UART driver role • Byte-at-a-time transmission • Transmit status • Ring buffer motivation • Ring buffer indices • Concurrent UART architecture • Blocking versus non-blocking APIs • Critical sections and interrupt safety • UART test cases • UART debugging workflow

08. SPI HAL and simulated device

SPI basics in the project • Initialization contract • Single-byte transfer • Buffer transfer • Chip select ownership • Clock mode and bit order • Simulated SPI device • SPI status handling • SPI test matrix • SPI troubleshooting

09. I2C HAL and simulated device

I2C bus model • Address conventions • Read and write operations • ACK and NACK • Timeout behavior • Simulation device behavior • I2C test matrix • I2C versus physical validation • Common I2C bugs • Extension exercise

10. OS abstraction layer

Why OSAL exists • Bare-metal backend • RTOS backend • API consistency • Task and ISR boundaries • Timing assumptions • Critical-section semantics • Backend selection • OSAL tests • Porting checklist

11. ROM and boot-flow modules

Boot-flow overview • Boot state machine • Firmware image metadata • Integrity versus authenticity • Signature module review • Trust anchor and key management • Rollback protection • Failure handling • Negative boot tests • Security review boundary

12. Firmware integrity and signature testing

Integrity test purpose • Known-answer tests • Mutation testing • Signature verification test • Input length validation • Failure status propagation • Test data hygiene • Reproducibility • Coverage limits • Documentation language

13. AXI4-Lite register map

Register map purpose • CONTROL register • STATUS register • OUTPUT register • Read access rules • Write access rules • Byte strobes • Reset values • Address alignment • Register-map change control

14. AXI4-Lite write channel

AW and W are independent • VALID and READY handshake • Pending address state • Pending data state • Write response channel • Backpressure testing • Invalid write targets • WSTRB corner cases • Write ordering • Write-channel debug checklist

15. AXI4-Lite read channel

AR and R channels • Read response stability • Mapped reads • Unmapped reads • Read-after-write behavior • Reset during reads • Read address acceptance • Read response code • Read test matrix • Waveform analysis

16. RTL simulation and testbench use

Compile a testbench • Run a compiled simulation • Functional testbench • Backpressure testbench • Reset testbench • Clock and reset discipline • Task timeout design • Assertions and checks • Waveform dumping • RTL regression checklist

17. C AXI driver and register access

Driver layering • Register definitions • Initialization state • Argument validation • Enable and disable • Output masking • Status reads • Mock register backend • Driver negative tests • PS-to-PL integration gap

18. Verification automation and logs

Purpose of the script • Fail-fast behavior • Required tool check • Log creation • Inspect the final log • Script exit code • CTest result interpretation • Report hygiene • CI extension • Automation review checklist

19. Git, GitHub, and release hygiene

Inspect before staging • Ignore generated files • Whitespace check • Small focused commits • README quality • License and attribution • Secrets and sensitive data • Tagging a release • Publishing a verification report • Public project claims

20. Troubleshooting and final acceptance

CMake cannot configure • Missing header or NULL • Undefined reference • CTest test missing • RTL compile error • RTL simulation hangs • Unexpected register value • Tests pass but confidence is low • Final acceptance checklist • Next engineering steps

Suggested first run

```
cd ~/Projects/secure_rom_hal
git status --short
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
./scripts/run_verification.sh
```

The final command assumes the script exists and is executable. Review the script and logs before relying on its success message.

What this repository is

The repository is a learning and portfolio framework that combines C-based platform utilities, peripheral HAL examples, OS abstraction, boot-flow-oriented modules, host-side tests, and a small custom AXI4-Lite peripheral in RTL. It is designed to demonstrate how software and programmable logic can be developed and verified in layers. It is not a production ROM implementation or a certified security product.

Engineering checklist

- Confirm the implementation in the current source file for: what this repository is.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “what this repository is” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

What the project demonstrates

The strongest demonstrated themes are modular C design, register access, driver API boundaries, negative testing, simulation models, build automation, and RTL protocol reasoning. The custom AXI4-Lite peripheral makes the software/hardware boundary concrete: C code describes register operations while RTL implements the register bank and bus responses.

Engineering checklist

- Confirm the implementation in the current source file for: what the project demonstrates.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “what the project demonstrates” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

What it does not prove

A project title containing “Secure ROM” does not by itself prove secure boot. A real secure boot chain needs reviewed cryptography, authenticated trust anchors, key lifecycle, rollback policy, fault handling, and threat-model-driven testing. This repository should be described as a learning framework with secure-boot-related modules unless those properties are independently implemented and validated.

Engineering checklist

- Confirm the implementation in the current source file for: what it does not prove.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “what it does not prove” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Repository map at a glance

The application entry point is under app/. Platform-level helpers live under platform/. Peripheral interfaces live under hal/. Simulation-only devices live under simulation/. OS abstraction code is under osal/. Boot and firmware-oriented modules live under rom/. RTL and testbenches are under rtl/. Host tests are under tests/. The verification script is under scripts/.

Engineering checklist

- Confirm the implementation in the current source file for: repository map at a glance.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “repository map at a glance” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

How to use this guide

Start with the quick start and tool prerequisites, then build and run the host tests. Read the driver and register chapters before studying the AXI RTL. Run each RTL testbench separately before using the automation script. Record actual command output and commit only source, documentation, and intentionally retained reports.

Engineering checklist

- Confirm the implementation in the current source file for: how to use this guide.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “how to use this guide” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Recommended reading order

A new contributor should first inspect README.md and CMakeLists.txt, then trace app/main.c. Next inspect platform register helpers and one simple HAL driver. Follow the same pattern for the AXI secure driver and its register map. Finally read the AXI RTL and the corresponding testbench together; the testbench is the executable statement of expected behavior.

Engineering checklist

- Confirm the implementation in the current source file for: recommended reading order.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “recommended reading order” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Development philosophy

Keep hardware-independent logic separate from memory-mapped register access. Make error paths explicit. Validate arguments before dereferencing pointers. Keep testable behavior small and deterministic. Prefer narrow APIs over exposing register details throughout the application. Each design claim should be paired with a test or clearly labeled as a design intention.

Engineering checklist

- Confirm the implementation in the current source file for: development philosophy.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “development philosophy” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Terminology used

HAL means hardware abstraction layer; OSAL means operating-system abstraction layer; MMIO means memory-mapped I/O; AXI4-Lite is a memory-mapped control interface; RTL means register-transfer-level logic; CTest is CMake's test runner; Icarus Verilog is used to compile and run Verilog testbenches in this project.

Engineering checklist

- Confirm the implementation in the current source file for: terminology used.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “terminology used” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Known project assumptions

The documented host workflow assumes a Linux shell, CMake, a C compiler, CTest, Icarus Verilog, and vvp. The project can be built and tested without a Zynq board because the described checks are host-side C tests and RTL simulations. A real PS-to-PL deployment would require a separate hardware platform, address map, driver adaptation, and board-level validation.

Engineering checklist

- Confirm the implementation in the current source file for: known project assumptions.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “known project assumptions” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Baseline evidence checklist

Before publishing, save the commit hash, build output, CTest summary, RTL PASS/FAIL messages, and script exit status. If a testbench only prints FAIL but exits zero, the shell script may not detect the failure. Treat the log as evidence only after searching it for failure tokens and checking the testbench exit behavior.

Engineering checklist

- Confirm the implementation in the current source file for: baseline evidence checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “baseline evidence checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Linux shell baseline

Use a standard Bash shell. Commands in this guide assume the repository is checked out at ~/Projects/secure_rom_hal. If your checkout uses another path, replace the path consistently. Do not run destructive Git cleanup commands until you understand whether the target files are tracked or locally modified.

Practical check

```
cmake --version
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “linux shell baseline” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Required toolchain

The verification script checks for cmake, ctest, iverilog, and vvp. A working C compiler is also required by CMake. On Ubuntu or Debian, the common packages are build-essential, cmake, and iverilog; CTest is normally supplied by CMake.

Practical check

```
ctest --version
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “required toolchain” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Install packages on Ubuntu

Install the build tools through the distribution package manager, then confirm each executable is available. Package names and versions can vary across distributions. Prefer system packages for a reproducible starter environment; use a project-specific toolchain only if the project later needs one.

Practical check

```
iverilog -V
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “install packages on ubuntu” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Check the compiler

Run `cc --version` and `gcc --version`. The project declares C11 in CMake, so the compiler must support C11. The exact compiler version is less important than a clean build and predictable diagnostics.

Practical check

```
vvp -V
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “check the compiler” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Check CMake and CTest

Run `cmake --version` and `ctest --version`. CMake configures targets and generates the build system; CTest discovers and executes registered tests. They are related but different stages, so a successful configure does not mean the test suite has passed.

Practical check

```
cc --version
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “check cmake and ctest” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Check Icarus Verilog

Run `iverilog -V` and `vvp -V`. Icarus compiles Verilog source and testbench files into a simulation executable; vvp runs that executable. Compilation success alone is not a functional simulation pass.

Practical check

```
git --version
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “check icarus verilog” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Verify the repository path

Use `cd ~/Projects/secure_rom_hal` followed by `pwd`. Then use `git status --short` to see local changes. If the directory is not a Git repository, clone the published repository or navigate to the existing checkout rather than creating a new repository over unrelated files.

Practical check

```
uname -a
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “verify the repository path” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Network-independent workflow

Once dependencies are installed, the host-side C and RTL simulation flow should be able to run without connecting to a board or cloud service. If a command tries to fetch dependencies, inspect CMake and scripts to understand why; do not assume network access is required unless the project configuration says so.

Practical check

```
pwd
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “network-independent workflow” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Permission and shell errors

If the script says permission denied, inspect its mode with `ls -l scripts/run_verification.sh` and use `chmod +x scripts/run_verification.sh`. If the shell says command not found, confirm installation and PATH before changing the script.

Practical check

```
git status --short
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “permission and shell errors” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Capture the environment

For a reproducible report, record `uname -a`, `cmake --version`, `cc --version`, `iverilog -V`, and the Git commit hash. Keep this information in a local verification note or release record; do not include usernames, private paths, or unrelated machine details unless useful.

Practical check

```
ls -la
```

Expected result: the command should run and report its version, environment, or repository state. If it fails, fix the environment before moving to the next build stage.

Review questions

- What observable behavior would demonstrate that “capture the environment” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Clone or open the repository

If hosted on GitHub, use `git clone <repository-url>` and then `cd secure_rom_hal`. If you already have a checkout, open it directly. Avoid copying the `build/` directory between machines because CMake build trees contain generated paths and machine-specific configuration.

Command reference

```
git clone <repository-url>
cd secure_rom_hal
```

Review questions

- What observable behavior would demonstrate that “clone or open the repository” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Inspect top-level files

Run `ls -la` and `find . -maxdepth 2 -type f | sort`. The top-level files should include CMakeLists.txt, README.md, application/source directories, tests, and RTL. The exact tree may evolve; use the current checkout as the source of truth.

Command reference

```
ls -la
find . -maxdepth 2 -type f | sort
```

Review questions

- What observable behavior would demonstrate that “inspect top-level files” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read CMakeLists.txt first

CMakeLists.txt declares the C standard, executable sources, include paths, and test targets. When a source file is added, ensure the relevant target includes it. When a header directory is needed, add the path to the appropriate target instead of relying on incidental include search paths.

Command reference

```
sed -n '1,220p' CMakeLists.txt
```

Review questions

- What observable behavior would demonstrate that “read cmakelists.txt first” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Configure a clean build

Run `cmake -S . -B build`. This separates source files from generated build artifacts. A configure error usually points to a missing tool, malformed CMake syntax, missing source path, or target configuration problem. Resolve configure failures before compiling.

Command reference

```
cmake -S . -B build
```

Review questions

- What observable behavior would demonstrate that “configure a clean build” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Compile the project

Run `cmake --build build`. This compiles the configured targets. Read the first meaningful compiler error, not just the final make failure. Common causes include missing includes, mismatched declarations, missing source files in CMake, and inconsistent function signatures.

Command reference

```
cmake --build build
```

Review questions

- What observable behavior would demonstrate that “compile the project” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Run CTest

Run `ctest --test-dir build --output-on-failure`. The `--output-on-failure` option shows details for failing tests. CTest only runs registered tests; a test executable that is built but not registered may not appear in the CTest summary.

Command reference

```
ctest --test-dir build --output-on-failure
```

Review questions

- What observable behavior would demonstrate that “run ctest” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Use verbose build output

When the build fails in an opaque way, run `cmake --build build --verbose`. This shows compiler and linker commands. It is useful for spotting include-path errors, missing source files, incorrect language flags, and unexpected build configuration.

Command reference

```
cmake --build build --verbose
```

Review questions

- What observable behavior would demonstrate that “use verbose build output” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Clean reconfigure safely

If CMake cache state is stale, create a fresh build directory such as `build-clean`: `cmake -S . -B build-clean`, then build it. This avoids deleting a directory whose contents you have not inspected. Generated directories should remain ignored by Git.

Command reference

```
cmake -S . -B build-clean
cmake --build build-clean
```

Review questions

- What observable behavior would demonstrate that “clean reconfigure safely” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

First-pass success criteria

A successful first pass means configuration completes, all expected targets build, and CTest reports no failures. This still does not establish that the RTL simulations pass or that the project works on physical hardware. Those are separate verification layers.

Command reference

```
ctest --test-dir build -N
```

Review questions

- What observable behavior would demonstrate that “first-pass success criteria” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Record your first baseline

Write down the commit hash, compiler version, CMake version, CTest summary, and any warnings. Warnings are useful engineering data. Do not silently suppress warnings simply to make the output look clean; investigate correctness-related diagnostics first.

Command reference

```
git rev-parse --short HEAD
```

Review questions

- What observable behavior would demonstrate that “record your first baseline” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CMake target model

Each executable target declares its own source list and include directories. This makes dependencies explicit and allows small tests to compile only the modules they need. It also means that a newly added source file must be connected to the correct target.

Engineering checklist

- Confirm the implementation in the current source file for: cmake target model.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “cmake target model” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

C standard selection

The project sets C11 as the required language standard. C11 supports common embedded-C constructs and gives a consistent baseline across compilers. If a feature is not available in the chosen toolchain, prefer portable alternatives unless a target-specific requirement justifies otherwise.

Engineering checklist

- Confirm the implementation in the current source file for: c standard selection.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “c standard selection” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Include directory discipline

Use target-specific include directories for platform, HAL, simulation, ROM, and OSAL headers. Duplicate include-directory entries do not normally break a build, but they are clutter and should be cleaned up during maintenance.

Engineering checklist

- Confirm the implementation in the current source file for: include directory discipline.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “include directory discipline” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Executable versus library

The current CMake structure uses executables for the main demo and tests. A larger production-style repository might package platform, HAL, OSAL, and ROM modules as libraries, but that refactor should preserve test coverage and not be done solely for appearance.

Engineering checklist

- Confirm the implementation in the current source file for: executable versus library.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “executable versus library” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Test target separation

Separate test executables make it easier to isolate a failing component. A ring-buffer test need not compile unrelated boot-flow code. A driver regression test should include the real driver implementation and the simulation or mock layer required by its contract.

Engineering checklist

- Confirm the implementation in the current source file for: test target separation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “test target separation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Linker diagnostics

An undefined reference usually means the function was declared but no matching definition was linked, or the definition has a different name/signature. Confirm the implementation file is non-empty, saved, and included in the target source list. Header declarations alone do not create function bodies.

Engineering checklist

- Confirm the implementation in the current source file for: linker diagnostics.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “linker diagnostics” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Compiler warning policy

Treat warnings as investigation prompts. A warning about assignment in a conditional, pointer/integer comparisons, or missing declarations may indicate real logic errors. Fix the cause rather than adding casts or suppressions that obscure the underlying defect.

Engineering checklist

- Confirm the implementation in the current source file for: compiler warning policy.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “compiler warning policy” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Build artifacts and Git

Generated build trees should not be committed. Use `.gitignore` for build directories and generated simulator binaries. If build files were already tracked, `.gitignore` alone does not untrack them; `git rm --cached` is needed, with care.

Engineering checklist

- Confirm the implementation in the current source file for: build artifacts and git.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “build artifacts and git” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Adding a new test

Add the test source, configure its target and include paths, then register it with CTest if it should run in the regression suite. Verify the test is actually listed by `ctest --test-dir build -N`. A binary existing in `build/` is not enough.

Engineering checklist

- Confirm the implementation in the current source file for: adding a new test.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “adding a new test” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Build-system review checklist

Check for missing sources, duplicate include paths, test targets not registered in CTest, inconsistent compile options, and generated artifacts tracked in Git. Keep the build reproducible and avoid relying on files that exist only on one developer's machine.

Engineering checklist

- Confirm the implementation in the current source file for: build-system review checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “build-system review checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Platform type layer

A platform type header centralizes fixed-width integer types and common definitions. Use `uint8_t`, `uint16_t`, `uint32_t`, and `uint64_t` when a width matters. Avoid assuming that a plain `int` has the exact width required by a hardware register.

Engineering checklist

- Confirm the implementation in the current source file for: platform type layer.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “platform type layer” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Status codes as API contracts

A shared status enum gives callers a consistent way to interpret success, invalid arguments, uninitialized state, busy state, and other failures. Callers should check return values instead of assuming a driver operation always succeeds.

Engineering checklist

- Confirm the implementation in the current source file for: status codes as api contracts.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “status codes as api contracts” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register read helper

A register read helper should make the access width and address interpretation explicit. For a host simulation, registers may be backed by a software array; on hardware, the implementation must use a valid MMIO mapping and target-specific ordering requirements.

Engineering checklist

- Confirm the implementation in the current source file for: register read helper.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “register read helper” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register write helper

A register write helper centralizes writes and makes testing easier. The host implementation can update a mock register map, while a target implementation may use volatile accesses or architecture-specific accessors. Do not assume a host array behaves exactly like a peripheral bus.

Engineering checklist

- Confirm the implementation in the current source file for: register write helper.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “register write helper” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Set and clear bit operations

A common set-bits operation is `reg |= mask`; clear-bits is `reg &= ~mask`. The mask should be of the intended width. Read-modify-write is not automatically atomic; shared registers may require interrupt protection, hardware set/clear registers, or a lock.

Engineering checklist

- Confirm the implementation in the current source file for: set and clear bit operations.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “set and clear bit operations” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Volatile is not synchronization

`volatile` can be important for memory-mapped registers because reads and writes must not be optimized away as ordinary memory. It does not make compound operations atomic and does not provide a thread-safe locking protocol.

Engineering checklist

- Confirm the implementation in the current source file for: volatile is not synchronization.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “volatile is not synchronization” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register address validation

A register abstraction should define which offsets are legal and how accesses are aligned. An allow-list can reject unknown offsets before touching a simulated or real register. Document whether offsets are byte addresses, word indices, or another representation.

Engineering checklist

- Confirm the implementation in the current source file for: register address validation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “register address validation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Endianness and width

A 32-bit register has a defined bit numbering in the software API, but byte ordering matters when data is serialized or accessed through byte lanes. State the expected endianness at interfaces and test partial writes explicitly where supported.

Engineering checklist

- Confirm the implementation in the current source file for: endianness and width.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “endianness and width” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Host model versus MMIO

Host tests often use ordinary arrays or mock functions to represent registers. This is valuable for logic testing but cannot validate bus timing, address decoding in a real interconnect, cache attributes, or the board's physical address map.

Engineering checklist

- Confirm the implementation in the current source file for: host model versus mmio.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “host model versus mmio” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Platform utility test plan

Test valid reads/writes, set/clear masks, invalid offsets, null pointers where relevant, and boundary values. Also test that failed operations leave state unchanged when that is part of the API contract.

Engineering checklist

- Confirm the implementation in the current source file for: platform utility test plan.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “platform utility test plan” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO API responsibilities

A GPIO HAL should expose operations such as initialize, write, read, and toggle without forcing application code to know the register layout. Keep pin validation and state validation inside the driver boundary.

Engineering checklist

- Confirm the implementation in the current source file for: gpio api responsibilities.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio api responsibilities” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO initialization

Initialization should establish a known software state and, for real hardware, configure direction and pin mux as required. A host simulation can demonstrate API behavior, but hardware pin mux and electrical configuration must be validated on the target board.

Engineering checklist

- Confirm the implementation in the current source file for: gpio initialization.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio initialization” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Pin range checks

If the supported pin count is 32, valid indices are typically 0 through 31. Use a range comparison such as `pin >= GPIO_PIN_COUNT`; avoid accidentally using assignment in a conditional. Shifts also require care because shifting by the type width is undefined in C.

Engineering checklist

- Confirm the implementation in the current source file for: pin range checks.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “pin range checks” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Write and read semantics

Define whether writing a pin updates a mock state, an output register, or a set/clear register. A read may represent output state or physical input state; those are not always identical. The API and tests should distinguish these meanings.

Engineering checklist

- Confirm the implementation in the current source file for: write and read semantics.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “write and read semantics” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Toggle implementation

A software toggle commonly reads the current state and writes the inverse. This is safe enough for a single-threaded mock, but can race if another context changes the pin between the read and write. Hardware toggle registers or synchronization may be preferable.

Engineering checklist

- Confirm the implementation in the current source file for: toggle implementation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “toggle implementation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO status handling

Return a status for invalid pin values or uninitialized drivers. Callers should be able to distinguish invalid input from successful low-level operation. Avoid silently ignoring errors that would make an application believe an output changed when it did not.

Engineering checklist

- Confirm the implementation in the current source file for: gpio status handling.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio status handling” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO test cases

Test a valid pin set high, set low, read after write, toggle high-to-low, toggle low-to-high, and invalid indices. Include boundary indices such as zero, the highest valid pin, and the first invalid pin.

Engineering checklist

- Confirm the implementation in the current source file for: gpio test cases.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio test cases” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO simulation boundaries

A host GPIO simulation cannot prove that a board LED lights, that an external voltage is correct, or that a pin is mapped to the intended package ball. Those require board files, constraints, pin-mux setup, and hardware measurements.

Engineering checklist

- Confirm the implementation in the current source file for: gpio simulation boundaries.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio simulation boundaries” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Common GPIO debugging

If the observed state is wrong, inspect the pin validation, mask creation, register offset, and whether a function uses `=` instead of `==`. Then inspect the caller's return-value handling. Avoid debugging the physical pin before the software state model is correct.

Engineering checklist

- Confirm the implementation in the current source file for: common gpio debugging.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “common gpio debugging” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

GPIO extension exercises

Add direction configuration, pull-up/pull-down options, edge callbacks, or a mock interrupt event. For each extension, define the API contract first and add negative tests before changing the implementation.

Engineering checklist

- Confirm the implementation in the current source file for: gpio extension exercises.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “gpio extension exercises” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

UART driver role

A UART HAL usually handles initialization, byte transmission, byte reception, and status reporting. A minimal host simulation may print bytes or store them in a buffer; a physical UART needs clock, baud-rate, pin mux, FIFO, and interrupt/DMA configuration.

Engineering checklist

- Confirm the implementation in the current source file for: uart driver role.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “uart driver role” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Byte-at-a-time transmission

A simple transmit loop writes one byte at a time. This is easy to understand but may be inefficient for high-throughput data because it can increase per-byte overhead and block the caller. Use it as a learning baseline, not as a throughput guarantee.

Engineering checklist

- Confirm the implementation in the current source file for: byte-at-a-time transmission.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “byte-at-a-time transmission” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Transmit status

A status return should communicate whether the byte was accepted, the peripheral was busy, or an argument was invalid. The caller should define whether it retries, waits, returns an error, or queues the data for later transmission.

Engineering checklist

- Confirm the implementation in the current source file for: transmit status.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “transmit status” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Ring buffer motivation

A ring buffer decouples producers and consumers. An application can enqueue data while a UART service routine drains it. Capacity, full/empty distinction, and concurrency rules must be explicit.

Engineering checklist

- Confirm the implementation in the current source file for: ring buffer motivation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ring buffer motivation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Ring buffer indices

A common ring buffer tracks read and write indices and wraps them at capacity. A one-empty-slot design sacrifices one element to distinguish full from empty; a count-based design stores an additional count but requires consistent updates.

Engineering checklist

- Confirm the implementation in the current source file for: ring buffer indices.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ring buffer indices” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Concurrent UART architecture

A concurrent UART wrapper can accept data into a queue and move it to the low-level transmitter when possible. The wrapper should define which context produces data, which context consumes it, and what happens when the queue is full.

Engineering checklist

- Confirm the implementation in the current source file for: concurrent uart architecture.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “concurrent uart architecture” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Blocking versus non-blocking APIs

A blocking write may wait until hardware can accept data. A non-blocking write returns quickly with a busy or partial-transfer status. Neither is universally better; the decision depends on latency requirements, caller context, and buffering.

Engineering checklist

- Confirm the implementation in the current source file for: blocking versus non-blocking apis.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “blocking versus non-blocking apis” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Critical sections and interrupt safety

A flag-only critical-section placeholder does not actually mask interrupts or provide mutual exclusion. On a target, use the platform's supported interrupt/lock primitive. Document that a host stub exists only to keep the example portable.

Engineering checklist

- Confirm the implementation in the current source file for: critical sections and interrupt safety.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “critical sections and interrupt safety” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

UART test cases

Test normal transmission, empty strings, null buffers if the API accepts pointers, buffer-full behavior, busy state, wrap-around, and repeated enqueue/dequeue cycles. For concurrency, include interleavings or stress tests where the test harness supports them.

Engineering checklist

- Confirm the implementation in the current source file for: uart test cases.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “uart test cases” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

UART debugging workflow

Trace data from application call to wrapper, queue, and low-level transmit function. Check return values at every layer. If output is missing, determine whether the byte was never enqueued, the queue was not drained, or the simulated peripheral did not accept the byte.

Engineering checklist

- Confirm the implementation in the current source file for: uart debugging workflow.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “uart debugging workflow” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

SPI basics in the project

SPI is a synchronous serial interface with clock, data-in, data-out, and chip-select behavior. A simple driver may expose initialization, one-byte transfer, and buffer transfer. Full-duplex semantics mean transmit and receive are related in each clocked transfer.

Engineering checklist

- Confirm the implementation in the current source file for: spi basics in the project.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “spi basics in the project” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Initialization contract

An initialization function should validate the configuration pointer before dereferencing it, set the peripheral state, and return a status. In C, `NULL` is defined through standard headers such as `<stddef.h>`; include the appropriate header rather than relying on incidental transitive includes.

Engineering checklist

- Confirm the implementation in the current source file for: initialization contract.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “initialization contract” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Single-byte transfer

A one-byte transfer should define how the received byte is returned and what happens when the receive pointer is null. If the API permits transmit-only or receive-only operation, document it explicitly and test those combinations.

Engineering checklist

- Confirm the implementation in the current source file for: single-byte transfer.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “single-byte transfer” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Buffer transfer

A buffer API should define whether zero length is legal, whether source and destination may alias, and whether partial completion is possible. If the driver is blocking, say so; if it is non-blocking, define how completion is reported.

Engineering checklist

- Confirm the implementation in the current source file for: buffer transfer.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “buffer transfer” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Chip select ownership

A device transaction often requires chip select to remain asserted across a sequence of bytes. If the HAL toggles chip select per byte, multi-byte device protocols may break. State whether the transfer function manages chip select or expects the caller to do so.

Engineering checklist

- Confirm the implementation in the current source file for: chip select ownership.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “chip select ownership” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Clock mode and bit order

SPI devices specify clock polarity, clock phase, and bit order. A simulated device should use the same assumptions as the driver. Do not claim interoperability with a real sensor until the correct mode, timing, and wiring are tested.

Engineering checklist

- Confirm the implementation in the current source file for: clock mode and bit order.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “clock mode and bit order” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Simulated SPI device

A host-side SPI device model provides deterministic responses for driver tests. It can validate control flow and expected bytes, but it does not reproduce electrical timing, setup/hold margins, signal integrity, or all device-specific edge cases.

Engineering checklist

- Confirm the implementation in the current source file for: simulated spi device.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “simulated spi device” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

SPI status handling

The project demo has shown normal TX status and busy TX status values in earlier runs. Confirm the current enum definitions before publishing numeric meanings. Prefer symbolic status names in logs and tests to avoid confusing status values with register contents.

Engineering checklist

- Confirm the implementation in the current source file for: spi status handling.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “spi status handling” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

SPI test matrix

Test initialization with valid and null configuration, a normal byte transfer, a null receive pointer, zero-length buffer behavior, a normal buffer transfer, simulated busy state, and error propagation from the device model.

Engineering checklist

- Confirm the implementation in the current source file for: spi test matrix.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “spi test matrix” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

SPI troubleshooting

When compilation reports `NULL` undeclared, include `<stddef.h>` or another standard header that defines it. If transfer data is wrong, inspect the selected SPI mode, byte order, pointer validation, and whether the mock device response matches the test's expected byte sequence.

Engineering checklist

- Confirm the implementation in the current source file for: spi troubleshooting.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “spi troubleshooting” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

I2C bus model

I2C uses a shared two-wire bus with addressing and start/stop conditions. A host model may represent a device by an address and a small register map, but it should not be mistaken for a cycle-accurate bus simulation.

Engineering checklist

- Confirm the implementation in the current source file for: i2c bus model.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “i2c bus model” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Address conventions

APIs differ in whether they accept a 7-bit address or a shifted 8-bit address byte. Choose one convention and state it in the header. A common integration bug is shifting an address twice or passing an address in the wrong format.

Engineering checklist

- Confirm the implementation in the current source file for: address conventions.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “address conventions” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read and write operations

A register write usually sends a device address, a register index, and one or more data bytes. A register read may require a repeated start between selecting the register and reading the data. The high-level API should make these steps clear.

Engineering checklist

- Confirm the implementation in the current source file for: read and write operations.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read and write operations” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

ACK and NACK

Acknowledgement tells the controller that a byte was accepted; NACK can indicate an absent device, a completed read, or another protocol condition depending on the phase. The API should map bus conditions to documented status values.

Engineering checklist

- Confirm the implementation in the current source file for: ack and nack.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ack and nack” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Timeout behavior

A peripheral can stall if the bus is held low or a device fails to respond. Timeout support prevents a caller from waiting forever. Use a monotonic time source or bounded polling count appropriate to the platform; do not rely on a host-only loop as a production timing guarantee.

Engineering checklist

- Confirm the implementation in the current source file for: timeout behavior.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “timeout behavior” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Simulation device behavior

A simulated I2C device should define supported addresses, readable/writable registers, reset values, and invalid access behavior. Keep the model deterministic so that failures can be reproduced.

Engineering checklist

- Confirm the implementation in the current source file for: simulation device behavior.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “simulation device behavior” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

I2C test matrix

Test a valid device address, unknown address, valid register read/write, boundary register index, null pointer where relevant, and timeout/error propagation. Test that a failed write does not accidentally modify a valid simulated register.

Engineering checklist

- Confirm the implementation in the current source file for: i2c test matrix.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “i2c test matrix” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

I2C versus physical validation

Host tests cannot prove correct pull-up resistor values, bus capacitance, clock stretching behavior, or board-level voltage levels. Those require hardware measurements and device-specific validation.

Engineering checklist

- Confirm the implementation in the current source file for: i2c versus physical validation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “i2c versus physical validation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Common I2C bugs

Frequent causes include address-format mismatch, incorrect repeated-start behavior, confusing byte count with register count, and failing to propagate a NACK or timeout. Trace the operation at the API, controller, and simulated device levels.

Engineering checklist

- Confirm the implementation in the current source file for: common i2c bugs.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “common i2c bugs” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Extension exercise

Add a temperature-sensor-style register map to the simulation and test a multi-byte read. Keep the simulation's scope explicit and avoid implying it is a model of a specific device unless the datasheet behavior is implemented.

Engineering checklist

- Confirm the implementation in the current source file for: extension exercise.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “extension exercise” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Why OSAL exists

An OS abstraction layer lets higher-level code use a stable interface while the implementation differs between bare-metal and RTOS environments. It reduces direct dependence on a specific scheduler or kernel API.

Engineering checklist

- Confirm the implementation in the current source file for: why osal exists.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “why osal exists” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Bare-metal backend

A bare-metal backend may implement delay, yield, critical-section, or task-like calls as simple functions or stubs. These are useful for host builds but must not be mistaken for a real scheduler or interrupt controller.

Engineering checklist

- Confirm the implementation in the current source file for: bare-metal backend.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “bare-metal backend” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

RTOS backend

An RTOS backend maps abstract operations to kernel services. The exact implementation depends on the selected RTOS, its version, and its configuration. Check that task priorities, stack sizes, and interrupt-context restrictions are respected.

Engineering checklist

- Confirm the implementation in the current source file for: rtos backend.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “rtos backend” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

API consistency

The public OSAL interface should have consistent types and error semantics across backends. If one backend silently ignores an operation while another blocks, that difference must be documented because it affects caller behavior.

Engineering checklist

- Confirm the implementation in the current source file for: api consistency.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “api consistency” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Task and ISR boundaries

Functions called from an interrupt handler must be safe in interrupt context. A queue operation that blocks is usually inappropriate inside an ISR. Separate ISR-safe enqueue operations from task-context processing when needed.

Engineering checklist

- Confirm the implementation in the current source file for: task and isr boundaries.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “task and isr boundaries” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Timing assumptions

A delay function may use ticks, cycles, or milliseconds. Its unit and resolution should be explicit. A host simulation cannot guarantee embedded real-time behavior because host scheduling and timing are different.

Engineering checklist

- Confirm the implementation in the current source file for: timing assumptions.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “timing assumptions” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Critical-section semantics

If the OSAL exposes critical-section functions, define nesting rules and whether interrupts, scheduler switching, or mutual exclusion are affected. A no-op host implementation should say so clearly.

Engineering checklist

- Confirm the implementation in the current source file for: critical-section semantics.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “critical-section semantics” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Backend selection

Build configuration should make the selected backend visible and avoid linking multiple competing implementations of the same API. If both bare-metal and RTOS source files are included in one host target, verify how duplicate definitions are prevented or conditionally compiled.

Engineering checklist

- Confirm the implementation in the current source file for: backend selection.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “backend selection” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

OSAL tests

Test return values, invalid inputs, and backend-specific behavior where possible. For concurrency primitives, include ordering and stress tests rather than relying only on a single happy-path example.

Engineering checklist

- Confirm the implementation in the current source file for: osal tests.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “osal tests” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Porting checklist

To port OSAL to another RTOS, map each abstract operation, define error mapping, check ISR safety, test timeout units, review stack and priority needs, and run the same high-level driver tests against the new backend.

Engineering checklist

- Confirm the implementation in the current source file for: porting checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “porting checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Boot-flow overview

The rom/ directory contains modules associated with boot flow, firmware image representation, integrity checks, signature-related logic, secure boot flow, and a ROM-facing API. Read their headers and source files together to determine which checks are implemented versus placeholders.

Engineering checklist

- Confirm the implementation in the current source file for: boot-flow overview.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “boot-flow overview” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Boot state machine

A boot flow is easier to reason about as explicit stages: initialize platform, locate image, validate metadata, check integrity/authenticity, and transfer control or report failure. Each transition should define inputs, outputs, and a failure path.

Engineering checklist

- Confirm the implementation in the current source file for: boot state machine.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “boot state machine” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Firmware image metadata

An image descriptor may carry a pointer, size, version, load address, or other metadata. Validate size and pointer bounds before parsing. Never trust an image-provided length without checking it against the actual available buffer.

Engineering checklist

- Confirm the implementation in the current source file for: firmware image metadata.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “firmware image metadata” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Integrity versus authenticity

A checksum or hash can detect accidental changes but does not prove who created the image. Authenticity requires a cryptographic signature or message authentication mechanism anchored to a trusted key. Keep these concepts separate in documentation and code review.

Engineering checklist

- Confirm the implementation in the current source file for: integrity versus authenticity.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “integrity versus authenticity” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Signature module review

Inspect the signature implementation to determine whether it uses a real cryptographic primitive or a simplified demonstration. Do not describe a placeholder or mock as cryptographic verification. A production design needs a reviewed library, key validation, error handling, and test vectors.

Engineering checklist

- Confirm the implementation in the current source file for: signature module review.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “signature module review” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Trust anchor and key management

A secure boot system needs a trust anchor whose integrity is protected, plus a key provisioning and rotation story. Private keys should not be embedded in a public source repository. Document the assumed root of trust and how the device rejects an untrusted image.

Engineering checklist

- Confirm the implementation in the current source file for: trust anchor and key management.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “trust anchor and key management” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Rollback protection

Authentic older firmware can still be vulnerable. Anti-rollback requires a monotonic version policy and a protected state source. If the project does not implement it, list it as a limitation and future extension rather than implying it is present.

Engineering checklist

- Confirm the implementation in the current source file for: rollback protection.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “rollback protection” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Failure handling

Boot failure should leave the system in a defined state: stop, enter recovery, select a known-good image, or report a specific error. Avoid ambiguous behavior that continues execution after validation fails.

Engineering checklist

- Confirm the implementation in the current source file for: failure handling.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “failure handling” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Negative boot tests

Test malformed headers, empty images, truncated images, invalid sizes, corrupted payloads, signature failures, and unsupported versions. Ensure failure tests verify both the returned status and the absence of an unintended state transition.

Engineering checklist

- Confirm the implementation in the current source file for: negative boot tests.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “negative boot tests” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Security review boundary

A tutorial-level ROM framework is not a substitute for a threat model, cryptographic review, fault-injection testing, secure key storage, or independent security assessment. Use precise language about what is implemented and what remains illustrative.

Engineering checklist

- Confirm the implementation in the current source file for: security review boundary.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “security review boundary” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Integrity test purpose

Integrity tests establish that a payload matches an expected digest or integrity value according to the implemented algorithm. The test should include both unchanged data and controlled corruption so that the expected failure path is exercised.

Engineering checklist

- Confirm the implementation in the current source file for: integrity test purpose.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “integrity test purpose” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Known-answer tests

A known-answer test compares the implementation's output against a published or independently generated expected value. This is stronger than comparing two calls to the same implementation, which may share the same defect.

Engineering checklist

- Confirm the implementation in the current source file for: known-answer tests.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “known-answer tests” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Mutation testing

Flip one bit in the payload, alter the metadata length, or corrupt a digest field and confirm that validation fails. Keep the test mutation deterministic so that a failure can be reproduced and diagnosed.

Engineering checklist

- Confirm the implementation in the current source file for: mutation testing.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “mutation testing” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Signature verification test

If a real signature verifier is present, use a valid test key and known signed data for a positive case, then test a modified message and a wrong public key for negative cases. Never put a private production key into the repository.

Engineering checklist

- Confirm the implementation in the current source file for: signature verification test.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “signature verification test” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Input length validation

Before hashing or verifying, check that the supplied pointer is valid and the declared size is within the available buffer. Ensure integer arithmetic used to compute end addresses cannot overflow.

Engineering checklist

- Confirm the implementation in the current source file for: input length validation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “input length validation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Failure status propagation

Validation failures should travel to the boot-flow caller as a documented error. Test that the caller does not treat a failed validation as a successful boot stage.

Engineering checklist

- Confirm the implementation in the current source file for: failure status propagation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “failure status propagation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Test data hygiene

Label test images and keys as synthetic. Avoid using proprietary firmware, classified material, customer images, or personal credentials in public examples.

Engineering checklist

- Confirm the implementation in the current source file for: test data hygiene.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “test data hygiene” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Reproducibility

Keep small deterministic test fixtures and document how they were generated. If a test requires a random nonce, seed or record it for reproducibility unless nondeterminism is itself under test.

Engineering checklist

- Confirm the implementation in the current source file for: reproducibility.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “reproducibility” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Coverage limits

A passing unit test set does not prove cryptographic correctness or resistance to side-channel attacks. Use tests to detect known functional failures, and use review and established cryptographic implementations for security-critical claims.

Engineering checklist

- Confirm the implementation in the current source file for: coverage limits.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “coverage limits” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Documentation language

Use “integrity-check path exercised” or “signature-related module tested” when that is what the evidence supports. Reserve “secure boot implemented” for a complete, reviewed, and validated chain with an explicit trust model.

Engineering checklist

- Confirm the implementation in the current source file for: documentation language.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “documentation language” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register map purpose

The custom AXI4-Lite peripheral uses a small memory-mapped register bank. The documented offsets are CONTROL at 0x0, STATUS at 0x4, and OUTPUT at 0x8. Confirm these constants in the current RTL and header before changing the design.

Register note

```
CONTROL  0x0
STATUS   0x4
OUTPUT   0x8
```

Review questions

- What observable behavior would demonstrate that “register map purpose” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CONTROL register

CONTROL bit 0 is used to enable or control the peripheral behavior in the current example. Other bits should be treated as reserved unless the RTL defines them. Software should avoid writing undocumented bits with arbitrary values.

Register note

```
CONTROL bit 0: enable/control (verify RTL)
```

Review questions

- What observable behavior would demonstrate that “control register” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

STATUS register

STATUS reflects the CONTROL[0] state in the described RTL. This is a simple status model, not a comprehensive health monitor. If the hardware later gains busy, error, or interrupt bits, define read-only semantics and reset values explicitly.

Register note

```
STATUS: reflects CONTROL[0] in the described design
```

Review questions

- What observable behavior would demonstrate that “status register” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

OUTPUT register

OUTPUT stores an output value that drives an 8-bit GPIO-style signal. The RTL uses byte strobes for writes, so software tests should confirm that only selected byte lanes update the register.

Register note

```
OUTPUT: 8-bit GPIO-style output
```

Review questions

- What observable behavior would demonstrate that “output register” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read access rules

The RTL returns a response for mapped addresses and a decode error for unsupported offsets, according to the described design. Test mapped reads, unmapped reads, and response stability when the receiver is not ready.

Register note

```
Read mapped offsets and check response code
```

Review questions

- What observable behavior would demonstrate that “read access rules” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Write access rules

The write path accepts address and data independently and combines them when both are available. Writes to STATUS and unmapped addresses are described as decode errors. Check the exact implementation to ensure the response code and register state behavior agree.

Register note

```
Write mapped offsets and inspect B response
```

Review questions

- What observable behavior would demonstrate that “write access rules” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Byte strobes

WSTRB indicates which bytes of the write data are valid. A correct byte-strobe implementation updates only the selected lanes. For an 8-bit output register, only the low byte may affect the output, but the register's storage and masking rules should be explicit.

Register note

```
WSTRB: selected byte lanes only
```

Review questions

- What observable behavior would demonstrate that “byte strobes” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Reset values

Document the reset values for CONTROL, STATUS-related state, and OUTPUT from the actual RTL. A reset test should verify both internal register state through bus reads and externally visible output signals where possible.

Register note

```
Assert reset; read back default state
```

Review questions

- What observable behavior would demonstrate that “reset values” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Address alignment

AXI4-Lite addresses are byte addresses. The current register offsets are word-spaced at 0x0, 0x4, and 0x8. Define how the design handles unaligned or unsupported offsets and make that behavior part of the test plan.

Register note

```
Check aligned word-spaced byte offsets
```

Review questions

- What observable behavior would demonstrate that “address alignment” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register-map change control

Treat register offsets and bit meanings as an interface contract between software and RTL. Update the header, RTL, driver, tests, and documentation together. A mismatch in any one layer can produce software that compiles but controls the wrong hardware.

Register note

```
Update RTL + header + driver + tests together
```

Review questions

- What observable behavior would demonstrate that “register-map change control” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

AW and W are independent

AXI4-Lite write address (AW) and write data (W) are separate channels. A master may present them in different cycles. A slave must not assume that AWVALID and WVALID are asserted together or that the channels are accepted simultaneously.

Engineering checklist

- Confirm the implementation in the current source file for: aw and w are independent.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “aw and w are independent” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

VALID and READY handshake

A transfer occurs on a rising clock edge when VALID and READY are both high. The sender must hold VALID and its payload stable until the handshake completes. A receiver can apply backpressure by deasserting READY.

Engineering checklist

- Confirm the implementation in the current source file for: valid and ready handshake.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “valid and ready handshake” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Pending address state

The example RTL tracks accepted address information using pending state so that an address accepted before its data can be retained. Inspect the pending flags and captured address registers to understand how the design supports independent channel arrival.

Engineering checklist

- Confirm the implementation in the current source file for: pending address state.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “pending address state” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Pending data state

The implementation should ensure that data is not applied until the matching address/data pair is available. Carefully reason about cases where AW arrives first, W arrives first, or both arrive together.

Engineering checklist

- Confirm the implementation in the current source file for: pending data state.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “pending data state” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Write response channel

After a write is processed, the slave presents a write response on BVALID and holds it until BREADY is asserted. The response code must remain stable while the response is stalled.

Engineering checklist

- Confirm the implementation in the current source file for: write response channel.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “write response channel” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Backpressure testing

The backpressure testbench should deliberately hold BREADY low and verify that BVALID and BRESP remain stable. It should also confirm that the write is not duplicated when the master finally accepts the response.

Engineering checklist

- Confirm the implementation in the current source file for: backpressure testing.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “backpressure testing” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Invalid write targets

The design rejects writes to STATUS and unmapped addresses with a decode error according to the current specification. Verify that rejected writes do not alter CONTROL, OUTPUT, or the externally visible GPIO value.

Engineering checklist

- Confirm the implementation in the current source file for: invalid write targets.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “invalid write targets” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

WSTRB corner cases

Test one selected byte, multiple selected bytes, no selected bytes, and reserved lanes. The expected behavior for zero strobes should be documented. The implementation should not accidentally update unselected bits.

Engineering checklist

- Confirm the implementation in the current source file for: wstrb corner cases.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “wstrb corner cases” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Write ordering

If a new write is presented while a response is pending, behavior must match the slave's buffering capacity. A simple single-outstanding design may apply backpressure; do not assume unlimited queued writes.

Engineering checklist

- Confirm the implementation in the current source file for: write ordering.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “write ordering” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Write-channel debug checklist

When a write test hangs, inspect AWVALID/AWREADY, WVALID/WREADY, captured payloads, pending flags, BVALID/BREADY, and reset. Use waveforms or targeted displays to identify the first handshake that fails.

Engineering checklist

- Confirm the implementation in the current source file for: write-channel debug checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “write-channel debug checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

AR and R channels

The read address channel uses ARVALID/ARREADY; the read data/response channel uses RVALID/RREADY. A read transfer begins when the address handshake completes and the slave later presents data and a response.

Engineering checklist

- Confirm the implementation in the current source file for: ar and r channels.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ar and r channels” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read response stability

When RVALID is high and RREADY is low, RDATA and RRESP must remain stable until the response is accepted. The read test should explicitly create this condition rather than relying on the receiver always being ready.

Engineering checklist

- Confirm the implementation in the current source file for: read response stability.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read response stability” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Mapped reads

Read CONTROL, STATUS, and OUTPUT and compare values with the expected state. Use a sequence that writes CONTROL or OUTPUT before reading back, then confirm that the returned value matches the documented register semantics.

Engineering checklist

- Confirm the implementation in the current source file for: mapped reads.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “mapped reads” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Unmapped reads

Read an unsupported offset and check the expected decode-error response. Ensure that the test does not mistake arbitrary data for a valid register value when the response indicates an error.

Engineering checklist

- Confirm the implementation in the current source file for: unmapped reads.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “unmapped reads” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read-after-write behavior

The testbench should wait for the write response before assuming the write is complete unless the interface contract permits a different ordering. Then issue a read and check that the updated state is visible.

Engineering checklist

- Confirm the implementation in the current source file for: read-after-write behavior.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read-after-write behavior” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Reset during reads

A reset test should verify how an in-flight read response is handled. Reset may clear VALID and restore defaults, but exact behavior must follow the RTL implementation and testbench expectations.

Engineering checklist

- Confirm the implementation in the current source file for: reset during reads.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “reset during reads” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read address acceptance

Check whether the slave can accept a new read address while an earlier response is pending. The answer depends on buffering in the RTL. Tests should not assume multiple outstanding reads if the implementation supports only one.

Engineering checklist

- Confirm the implementation in the current source file for: read address acceptance.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read address acceptance” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read response code

RRESP indicates whether the access succeeded or failed. The software-facing driver may not directly observe AXI response codes in a host mock, so RTL tests and C tests validate different layers of the contract.

Engineering checklist

- Confirm the implementation in the current source file for: read response code.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read response code” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Read test matrix

Include all mapped offsets, unmapped offsets, reads before initialization if applicable, backpressured R channel, reset interactions, and repeated reads. Verify both data and response code.

Engineering checklist

- Confirm the implementation in the current source file for: read test matrix.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “read test matrix” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Waveform analysis

In a waveform, start at the rising clock edge and mark each VALID/READY handshake. Follow address capture, data selection, response assertion, and response acceptance. This avoids guessing based on signal levels between edges.

Engineering checklist

- Confirm the implementation in the current source file for: waveform analysis.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “waveform analysis” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Compile a testbench

A typical Icarus command is `iverilog -g2012 -o build/verification/axi_functional rtl/axi_secure_peripheral.v rtl/tb/tb_axi_secure_peripheral.v`. The `-g2012` flag enables SystemVerilog-compatible language features accepted by Icarus, even when the design is primarily Verilog.

Engineering checklist

- Confirm the implementation in the current source file for: compile a testbench.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “compile a testbench” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Run a compiled simulation

Run `vvp build/verification/axi_functional`. Review all output for explicit PASS, FAIL, ERROR, or FATAL messages. An exit code of zero is useful but not sufficient if the testbench only prints failures and does not call `$fatal` or return a nonzero status.

Engineering checklist

- Confirm the implementation in the current source file for: run a compiled simulation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “run a compiled simulation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Functional testbench

The functional testbench should exercise normal register reads and writes, enable/control behavior, output behavior, and invalid address cases as implemented. Read the testbench tasks to understand the order of transactions and what constitutes a pass.

Engineering checklist

- Confirm the implementation in the current source file for: functional testbench.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “functional testbench” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Backpressure testbench

The backpressure testbench should force the receiver to withhold READY and confirm that the slave holds response information stable. It should test both the write response and read response paths if those paths are included in the testbench.

Engineering checklist

- Confirm the implementation in the current source file for: backpressure testbench.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “backpressure testbench” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Reset testbench

The reset testbench should assert reset, verify defaults, perform a transaction, reset again, and verify state restoration. It should also consider reset while channel handshakes are pending if the testbench includes that scenario.

Engineering checklist

- Confirm the implementation in the current source file for: reset testbench.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “reset testbench” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Clock and reset discipline

A testbench clock should have a predictable period and reset should be asserted for enough time to reach a clock edge. Avoid changing control signals at the exact sampling edge without understanding simulator scheduling, which can create race-like behavior.

Engineering checklist

- Confirm the implementation in the current source file for: clock and reset discipline.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “clock and reset discipline” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Task timeout design

Each transaction task should have a bounded wait for handshake completion. A global simulation timeout is helpful but can make failures harder to diagnose. On timeout, print the task name and relevant channel signals.

Engineering checklist

- Confirm the implementation in the current source file for: task timeout design.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “task timeout design” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Assertions and checks

Prefer explicit comparisons and `$fatal` on failed expectations. Assertions can make protocol properties clearer, but ensure the chosen simulator supports the syntax. Check that every test case has a pass/fail result and that the final process exit status reflects failures.

Engineering checklist

- Confirm the implementation in the current source file for: assertions and checks.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “assertions and checks” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Waveform dumping

If waveform dumping is enabled, open the generated VCD in a waveform viewer and inspect signal timing. Keep large generated waveform files out of Git unless they are intentionally included as a small illustrative artifact.

Engineering checklist

- Confirm the implementation in the current source file for: waveform dumping.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “waveform dumping” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

RTL regression checklist

Compile each testbench independently, run it, inspect output, confirm nonzero exit on failure, and then run the automation script. Do not count a test as passed just because it compiled.

Engineering checklist

- Confirm the implementation in the current source file for: rtl regression checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “rtl regression checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Driver layering

The AXI secure driver exposes a software API for the custom register block. Its responsibilities include checking arguments, managing initialization state, and converting higher-level requests into register reads/writes. The RTL remains responsible for bus-level handshakes.

Engineering checklist

- Confirm the implementation in the current source file for: driver layering.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “driver layering” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Register definitions

The register header centralizes offsets and bit definitions. Avoid duplicating magic numbers in application code. If an offset changes, update the shared header and RTL together, then run both C and RTL regressions.

Engineering checklist

- Confirm the implementation in the current source file for: register definitions.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “register definitions” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Initialization state

If the driver tracks an initialized flag or base address, every public operation should check it before accessing registers. Test calls made before initialization and verify a documented error is returned.

Engineering checklist

- Confirm the implementation in the current source file for: initialization state.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “initialization state” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Argument validation

Pointer-based APIs should validate required pointers before dereferencing. Decide whether optional receive pointers are supported or must be non-null. Negative tests should cover each invalid argument path.

Engineering checklist

- Confirm the implementation in the current source file for: argument validation.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “argument validation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Enable and disable

Enable/disable operations usually manipulate a documented control bit. They should preserve unrelated bits if those bits are defined. A driver should not write arbitrary values to reserved bits unless the register contract says to do so.

Engineering checklist

- Confirm the implementation in the current source file for: enable and disable.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “enable and disable” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Output masking

The example output is 8 bits wide. The driver should mask values to the supported width or reject out-of-range values according to the API contract. Tests should confirm the chosen behavior with values such as 0x00, 0xFF, and 0x1FF.

Engineering checklist

- Confirm the implementation in the current source file for: output masking.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “output masking” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Status reads

A status API should return the current status register or decode it into a stable software representation. Keep read errors distinct from a valid status value so that the caller cannot confuse a failed read with zero.

Engineering checklist

- Confirm the implementation in the current source file for: status reads.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “status reads” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Mock register backend

A host mock allows the driver to be tested without physical MMIO. Ensure the mock has the same offsets, width, and access restrictions as the RTL. A mock that accepts every address can hide bugs that the RTL rejects.

Engineering checklist

- Confirm the implementation in the current source file for: mock register backend.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “mock register backend” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Driver negative tests

Test null pointers, uninitialized driver use, unsupported register access, invalid values, and failed register operations where the mock can inject failures. Check both return codes and state after failure.

Engineering checklist

- Confirm the implementation in the current source file for: driver negative tests.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “driver negative tests” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

PS-to-PL integration gap

Host driver tests validate software logic against the mock, not a real Zynq processing system connected to programmable logic. Real integration needs the correct physical base address, AXI interconnect configuration, device-tree or bare-metal mapping, cache/MMU treatment, and board-level tests.

Engineering checklist

- Confirm the implementation in the current source file for: ps-to-pl integration gap.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ps-to-pl integration gap” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Purpose of the script

The `scripts/run_verification.sh` script aims to provide a repeatable one-command flow: configure CMake, build C targets, run CTest, compile RTL testbenches, run simulations, and capture output in `reports/final_verification.log`. Inspect the current script before relying on this exact sequence.

Verification command

```
chmod +x scripts/run_verification.sh
```

Review questions

- What observable behavior would demonstrate that “purpose of the script” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Fail-fast behavior

With `set -Eeuo pipefail`, shell failures should stop the script, including failures in pipelines. This is useful for build errors, but it does not automatically convert a printed testbench FAIL into a nonzero process exit.

Verification command

```
./scripts/run_verification.sh
```

Review questions

- What observable behavior would demonstrate that “fail-fast behavior” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Required tool check

The script checks for cmake, ctest, iverilog, and vvp before starting. If a tool is missing, install it or adjust the documented prerequisites. Avoid skipping a check silently because it makes the log appear complete when a stage did not run.

Verification command

```
grep -nEi 'PASS|FAIL|ERROR|FATAL' reports/final_verification.log
```

Review questions

- What observable behavior would demonstrate that “required tool check” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Log creation

The script creates the reports directory and resets the final verification log at the beginning. That means each run overwrites the previous log unless the script is changed. Save prior logs elsewhere if you need a historical comparison.

Verification command

```
tail -n 80 reports/final_verification.log
```

Review questions

- What observable behavior would demonstrate that “log creation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Inspect the final log

Use `grep -nEi 'PASS|FAIL|ERROR|FATAL' reports/final_verification.log` to locate notable test messages. Also inspect the end of the log and confirm every expected stage appears. A PASS token can be printed by one test while another later test fails.

Verification command

```
echo $?
```

Review questions

- What observable behavior would demonstrate that “inspect the final log” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Script exit code

After the script runs, inspect `$?` immediately or rely on the shell's visible success/failure result. Do not run another command first if you need the exit code of the verification script.

Verification command

```
ctest --test-dir build --output-on-failure
```

Review questions

- What observable behavior would demonstrate that “script exit code” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CTest result interpretation

CTest summarizes registered tests and their statuses. A passing CTest suite does not automatically include RTL testbenches unless they are explicitly integrated into CTest; this project's script runs RTL simulations as separate steps.

Verification command

```
ctest --test-dir build -N
```

Review questions

- What observable behavior would demonstrate that “ctest result interpretation” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Report hygiene

Commit a concise verification summary if it is useful, but avoid committing massive generated logs or environment-specific output unless you intentionally want them as release evidence. Keep raw logs reproducible and consider a small summary file.

Verification command

```
git status --short
```

Review questions

- What observable behavior would demonstrate that “report hygiene” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CI extension

The same steps can be adapted to a continuous-integration workflow. Pin tool versions where reproducibility matters, run on a clean checkout, archive logs as build artifacts, and fail the workflow when any test stage fails.

Verification command

```
git diff --check
```

Review questions

- What observable behavior would demonstrate that “ci extension” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Automation review checklist

Confirm executable permissions, required-tool checks, fail-fast behavior, log completeness, correct testbench source paths, nonzero failure propagation, and an accurate success message. Update the script whenever a new testbench or CTest target is added.

Verification command

```
git log --oneline -5
```

Review questions

- What observable behavior would demonstrate that “automation review checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Inspect before staging

Use `git status --short` to review changed, deleted, and untracked files. Untracked reports and generated files should be reviewed individually rather than staged blindly. Use `git diff --stat` and `git diff --check` to inspect the change set.

Engineering checklist

- Confirm the implementation in the current source file for: inspect before staging.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “inspect before staging” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Ignore generated files

The `.gitignore` should exclude build directories, object files, simulator binaries, and other generated artifacts. It should not hide source files, testbenches, scripts, or useful documentation. If an artifact is already tracked, add it to ignore rules and untrack it deliberately.

Engineering checklist

- Confirm the implementation in the current source file for: ignore generated files.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ignore generated files” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Whitespace check

Run `git diff --check` before committing. It detects trailing whitespace and certain patch formatting issues. Clean the source and stage the fix rather than ignoring the warning.

Engineering checklist

- Confirm the implementation in the current source file for: whitespace check.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “whitespace check” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Small focused commits

Use commit messages that describe one logical change, such as documentation, test automation, or driver behavior. Small commits make it easier to review regressions and identify which change introduced a failure.

Engineering checklist

- Confirm the implementation in the current source file for: small focused commits.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “small focused commits” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

README quality

The README should quickly explain purpose, features, build commands, test commands, repository structure, limitations, and next steps. Detailed operational procedures belong in the full guide so the README remains approachable.

Engineering checklist

- Confirm the implementation in the current source file for: readme quality.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “readme quality” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

License and attribution

Choose a license only if you have the rights to publish the code and any included third-party material. A license file should match the intended permissions. Do not copy vendor code, customer material, confidential details, or proprietary firmware into a public repository.

Engineering checklist

- Confirm the implementation in the current source file for: license and attribution.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “license and attribution” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Secrets and sensitive data

Search for credentials, private keys, API tokens, internal server names, customer data, and proprietary paths before pushing. Do not publish real signing keys or confidential defense-project details. Use synthetic examples and public-safe descriptions.

Engineering checklist

- Confirm the implementation in the current source file for: secrets and sensitive data.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “secrets and sensitive data” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Tagging a release

After a clean verification run, create a version tag only if you intend to maintain that release. A tag should point to a tested commit and should be accompanied by a concise release note listing included tests and known limitations.

Engineering checklist

- Confirm the implementation in the current source file for: tagging a release.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “tagging a release” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Publishing a verification report

A report should state the commit hash, environment, command sequence, actual pass/fail summary, and limitations. Do not write “all tests passed” unless you have fresh output supporting that claim.

Engineering checklist

- Confirm the implementation in the current source file for: publishing a verification report.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “publishing a verification report” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Public project claims

Describe this as a host-tested C/HAL and simulated AXI4-Lite learning framework. Do not claim silicon validation, timing closure, cryptographic certification, or real PS-to-PL integration unless those activities were actually performed and recorded.

Engineering checklist

- Confirm the implementation in the current source file for: public project claims.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “public project claims” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CMake cannot configure

Check that CMakeLists.txt exists and starts with `cmake_minimum_required` and a valid `project()` declaration. Verify every source path named in a target exists. Read the first CMake error and fix that exact issue before trying unrelated changes.

Engineering checklist

- Confirm the implementation in the current source file for: cmake cannot configure.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “cmake cannot configure” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Missing header or NULL

For `NULL` undeclared, include `<stddef.h>` in the source file that uses it. For a project header not found, check its filename, include spelling, target include directories, and capitalization. Linux filesystems are usually case-sensitive.

Engineering checklist

- Confirm the implementation in the current source file for: missing header or null.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “missing header or null” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Undefined reference

Confirm that the source file contains the function definition, that it was saved, and that the target links that source file. Also compare the declaration and definition for name, parameter types, and C/C++ linkage.

Engineering checklist

- Confirm the implementation in the current source file for: undefined reference.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “undefined reference” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

CTest test missing

Run `ctest --test-dir build -N`. If a test executable is not registered, check whether CMake calls `enable_testing()` and `add_test()` appropriately. Building a test executable does not automatically make CTest run it.

Engineering checklist

- Confirm the implementation in the current source file for: ctest test missing.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “ctest test missing” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

RTL compile error

Check module names, source order, syntax, and language mode. Include the RTL module and its testbench in the compile command. A testbench may use SystemVerilog constructs that require a suitable Icarus language flag.

Engineering checklist

- Confirm the implementation in the current source file for: rtl compile error.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “rtl compile error” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

RTL simulation hangs

Inspect the first expected handshake that never completes. Add bounded waits to tasks and print channel signals on timeout. Check whether READY is held low, whether reset remains asserted, and whether the testbench drives signals at safe times relative to the clock.

Engineering checklist

- Confirm the implementation in the current source file for: rtl simulation hangs.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “rtl simulation hangs” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Unexpected register value

Check register offset constants, reset values, write strobes, address/data channel ordering, and whether the driver mock matches the RTL. Verify whether the read response indicates an error before interpreting RDATA.

Engineering checklist

- Confirm the implementation in the current source file for: unexpected register value.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “unexpected register value” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Tests pass but confidence is low

Review negative cases, boundary values, error propagation, reset behavior, backpressure, and failure exit codes. A green test summary is meaningful only to the extent that the tests check the intended properties.

Engineering checklist

- Confirm the implementation in the current source file for: tests pass but confidence is low.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “tests pass but confidence is low” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Final acceptance checklist

Confirm clean build, CTest results, functional RTL simulation, backpressure simulation, reset simulation, log inspection, clean Git status, accurate README, no secrets, and a clear limitations section. Save the exact commit that was tested.

Engineering checklist

- Confirm the implementation in the current source file for: final acceptance checklist.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “final acceptance checklist” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Next engineering steps

Potential improvements include stronger protocol assertions, transaction-level coverage, bounded waits in every testbench task, a reusable MMIO backend, a real cryptographic library for signature verification if appropriate, CI automation, and a documented board-integration plan. Treat each as a separate, test-driven milestone.

Engineering checklist

- Confirm the implementation in the current source file for: next engineering steps.
- Check the normal path and at least one negative or boundary case.
- Check that errors are returned or reported rather than silently ignored.
- Record what the test proves and what remains outside its scope.

Review questions

- What observable behavior would demonstrate that “next engineering steps” works as intended?
- Which test would fail if the interface contract were accidentally changed?
- Does this check exercise host software, simulated RTL, or physical hardware?

Appendix A — Command quick reference

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
ctest --test-dir build -N
iverilog -g2012 -o build/verification/axi_functional rtl/axi_secure_peripheral.v rtl/tb/tb_axi_secure_peripheral.v
vvp build/verification/axi_functional
grep -nEi 'PASS|FAIL|ERROR|FATAL' reports/final_verification.log
git status --short
git diff --check
git log --oneline -5
```

Use the exact paths and target names present in your checkout. The script may evolve; treat its current contents as authoritative.

Appendix B — Public release checklist

- Build from a clean checkout and save the tested commit hash.
- Run CTest and every RTL testbench; inspect output and exit status.
- Confirm the script stops on failures and testbench failures propagate to the process exit code.
- Remove build products and simulator binaries from Git history where practical.
- Review all code, reports, and documentation for confidential or proprietary material.
- Do not publish private signing keys, tokens, internal network details, or customer data.
- State clearly that host mocks are not physical hardware validation.
- State clearly that educational signature/integrity modules are not a security certification.
- Ensure README build commands match the current CMake configuration.
- Keep a short release note with verified tests and known limitations.